Deep Reinforcement Learning

Temporal Difference Learning & Q Learning

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- The TD learning concept
- Prediction using TD learning
- Policy improvement / control using TD learning
  - SARSA
  - Q-learning

The TD learning concept

Temporal-difference learning and the previous methods

The main aspects of our previous two categories of methods:

1. Monte Carlo (MC): **learning from experience**, possibly without knowledge of a model, e.g., *MC prediction* with exploring starts:

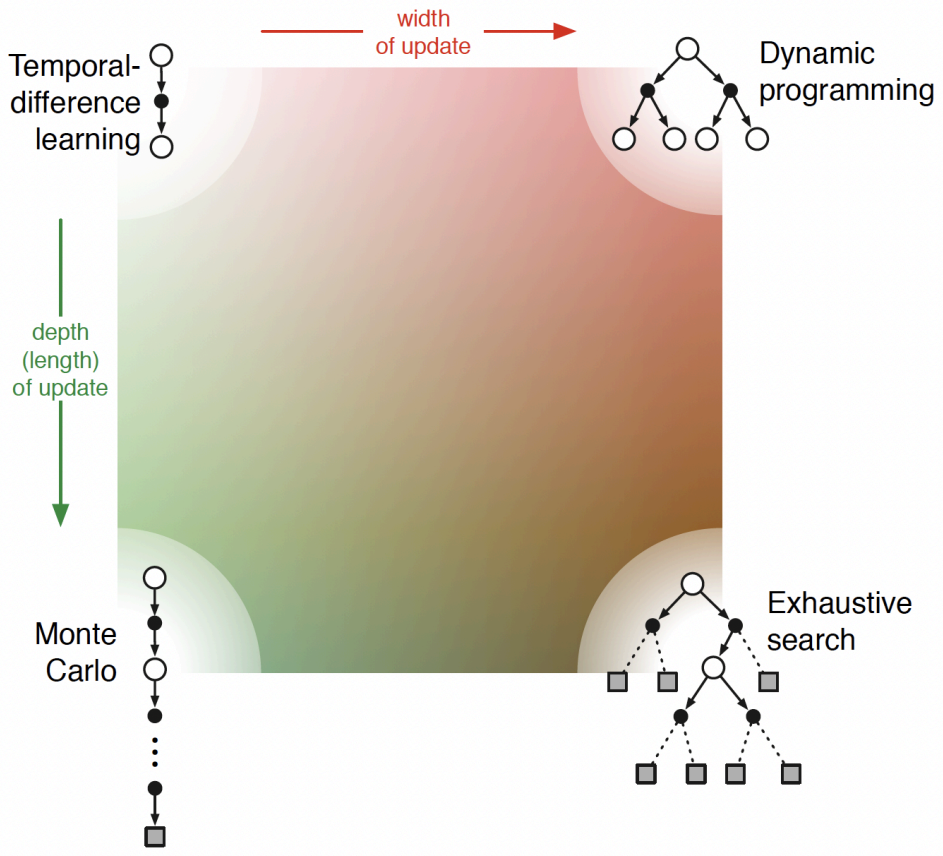
$$Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{n(s_t, a_t)}\left[g - Q(s_t, a_t)\right].$$

2. Dynamic programming (DP): **bootstrapping** ⇒ updating estimates based on estimates, e.g., *iterative policy evaluation*:

$$V(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} p(s'|s, a)\left[r + \gamma V(s')\right].$$

The **concept of temporal difference (TD) learning** is to combine both:

1. *Model-free* prediction and control in unknown MDPs.
2. Updates policy evaluation and improvement in an online fashion (i.e., not per episode) by *bootstrapping*.



(*Sutton and Barto 1998*, *Figure 8.11*)

💡 We are still considering finite MDPs here.

Prediction using TD learning

The general TD prediction update

Recall the **incremental update** of the type we discussed for multi-armed bandits or for MC control $\Rightarrow$ **NewEstimate** $\leftarrow$ **OldEstimate** $+ \mathit{StepSize}$ [**Target** $-$ **OldEstimate**]:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[g_t - V(s_t) \right]. \tag{1}$$

- $\alpha \in (0, 1)$ is the forget factor / step size.
- g_t is the target of the incremental update rule.
- To execute (1), we have to wait until the end of the episode to get g_t.

One-step TD / TD(0) update

$$V(s_t) \leftarrow V(s_t) + \alpha \left[r_t + \gamma V(s_{t+1}) - V(s_t) \right]. \tag{2}$$

- Here, the **TD target** is $r_t + \gamma V(s_{t+1})$.
- TD is *bootstrapping*: estimate $V(s_t)$ based on $V(s_{t+1})$.
- *Delay time of one* time step; no need to wait until the end of the episode.

Algorithmic implementation: TD-based prediction

Algorithm: TD-based prediction.
Parameters: Step size $\alpha \in (0, 1)$

Initialize: $V(s)$ arbitrarily for $s \in \mathcal{S}$, $V(\textbf{terminal}) = 0$

for $k = 1, 2, \ldots, K$ episodes:
    for $t = 0, 1, \ldots, T$:
        Sample action $a_t \sim \pi(a|s)$ and apply
        Observe s_{t+1} and r_t
        $V(s_t) \leftarrow V(s_t) + \alpha\left[r_t + \gamma V(s_{t+1}) - V(s_t)\right]$ (Eq. (2))
        if s_{t+1} is terminal **then** STOP

💡 The algorithm can directly be applied to the prediction of action-value functions.

The TD error and its relation to the MC error

Using the target, we can define the **TD error*** $\delta \Rightarrow$ the expression inside the bracket of Eq. (2) we're using to improve our estimate:

$$\delta_t = \textbf{target} - V(s_t) = r_t + \gamma V(s_{t+1}) - V(s_t).$$

Batch mode: Let's assume that the TD(0) estimate of $V(s)$ does not change within an episode, but that we apply all updates simulatenously once the episode is finished (exactly as we need to do in MC prediction):

$$\begin{aligned}
\underbrace{g_t - V(s_t)}_{\text{MC error}} &= r_t + \gamma g_{t+1} - V(s_t) + \gamma V(s_{t+1}) - \gamma V(s_{t+1}) \\
&= \delta_t + \gamma(\underbrace{g_{t+1} - V(s_{t+1})}_{\text{MC error}}) = \delta_t + \gamma \delta_{t+1} + \gamma^2 (g_{t+2} - V(s_{t+2})) \\
&= \delta_t + \gamma \delta_{t+1} + \gamma^2 \delta_{t+2} + \gamma^3 (g_{t+3} - V(s_{t+3})) = \ldots = \sum_{k=t}^{T} \gamma^{k-t} \delta_k.
\end{aligned}$$

- The MC error is the discounted sum of TD errors in this case.
- If $V(s)$ is updated during an episode (as is common in TD(0)), the above identity only holds approximately.

* For sufficiently small step size α, it can be shown that the TD error conveges to zero in batch mode (Sutton and Barto 1998).

Convergence of TD(0)

Given a finite MDP and a fixed policy π, the state-value estimate V of TD(0) converges to the true V^π (that is, $\lim_{t\to\infty} \delta_t = 0$)

- in the mean for a constant but sufficiently small step-size α and
- with probability one if the step-size holds the *Robbins-Munro* condition

$$\sum_{t=1}^{\infty} \alpha_t = \infty \qquad \text{and} \qquad \sum_{t=1}^{\infty} \alpha_t^2 < \infty. \tag{3}$$

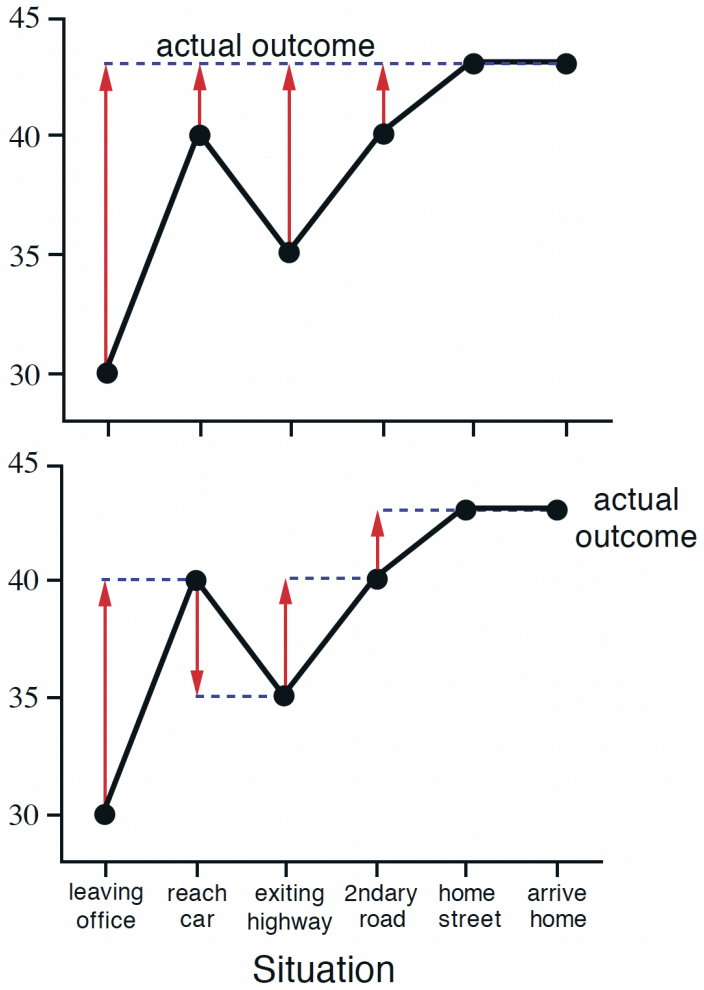
- In particular, $\alpha_k = \frac{1}{k}$ meets the condition (3).
- TD(0) often converges faster than MC, as we will see in the following examples. However, there is no guarantee it's faster.
- TD(0) can be more sensitive to poor initializations $V_0(s)$ compared to MC.

The *driving home* example

Imagine you want to predict your time to get home after work, starting with an estimate of 30 minutes.

State	Elapsed Time (minutes)	Predicted Time to Go	Predicted Total Time
leaving office, friday at 6	0	30	30
reach car, raining	5	35	40
exiting highway	20	15	35
2ndary road, behind truck	30	10	40
entering home street	40	3	43
arrive home	43	0	43

Overview of elapsed time and your predictions of time to go and total time (Sutton and Barto 1998, Example 6.1)



MC vs. TD estimate

- In MC, we estimate the returns g_t **after** we have reached home ⇒ it is hard to assess where the deviation was caused.
- In TD, we can immediately shift our estimate in each time step.

- With TD, we can even learn from continuing tasks!

The batch training AB example (1)

Assume that we have an MDP with only two states: A and B. We do not consider discounting (i.e., $\gamma = 1$).

We collect eight samples from the dynamics, and we would like to estimate the values $V(A)$ and $V(B)$:

Episode k	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Sequence (s, r, s, r)	A, 0, B, 0	B, 1	B, 1	B, 1	B, 1	B, 1	B, 1	B, 0

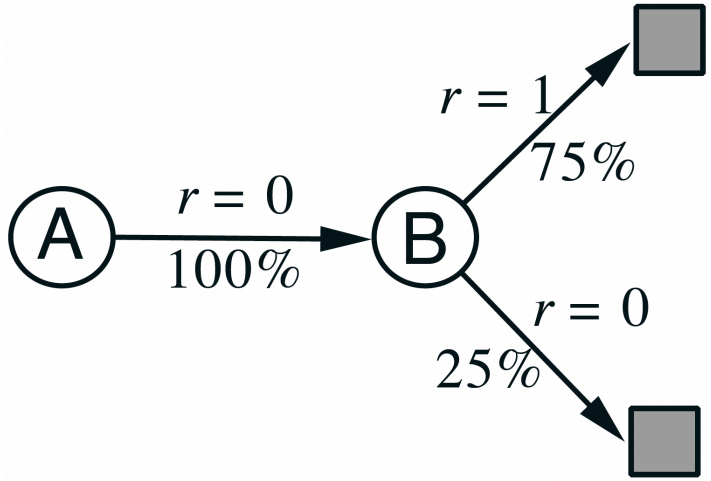
Sample of $K = 8$ trajectories, including the rewards following the states.

Question: What's $V(A)$ and $V(B)$ based on

- the MC estimate?
- the TD(0) estimate?

Before approaching this, let's try to model the underlying MDP!

- B occurs eight times, follwed by $r = +1$ in six, and by $r = 0$ in two cases.
  $\Rightarrow V(B) = \frac{3}{4}$.
- What about $V(A)$?
  1. In our example, the only episode containing A has return $g = 0$ $\quad \Rightarrow V(A) = 0$.
  2. On the other hand, A is followed by $r = 0$ and then $s = B$ $\quad \Rightarrow V(A) = V(B) = \frac{3}{4}$.



Option 2. for modeling the MDP (Sutton and Barto 1998, Ex. 6.4)

Answer 1. is the MC estimate, answer 2. is the TD estimate!

The batch training AB example (2)

Let's formally calculate the MC and TD updates:

$$V(s_t) \leftarrow V(s_t) + \alpha\left[g_t - V(s_t)\right] \qquad \text{(MC)}$$
$$V(s_t) \leftarrow V(s_t) + \alpha\left[r_t + \gamma V(s_{t+1}) - V(s_t)\right] \qquad \text{(TD)}$$

Convergence: no change in the estimate:

$$\alpha\left[g_t - V(s_t)\right] = g_t - V(s_t) = 0 \qquad \text{(MC)}$$
$$\alpha\left[r_t + \gamma V(s_{t+1}) - V(s_t)\right] = r_t + \gamma V(s_{t+1}) - V(s_t) = 0 \qquad \text{(TD)}$$

Consider a batch sweep over the the K episodes:

$$\sum_{k=1}^{K} g_{t,k} - V(s_{t,k}) = 0 \qquad \text{(MC)}$$
$$\sum_{k=1}^{K} r_{t,k} + \gamma V(s_{t+1,k}) - V(s_{t,k}) = 0 \qquad \text{(TD)}$$

The batch training AB example (3)

Apply the previous equations first to **state** B. Since B is a terminal state, $V(s_{t+1}) = 0$ and $g_{t,k} = r_{t,k}$ apply, i.e., the MC and TD updates are identical for B:

$$\text{MC}\Big|_{s=B}: \qquad \sum_{k=1}^{K} g_{t,k} - V(B) = 0 \qquad \Leftrightarrow \qquad V(B) = \frac{1}{K}\sum_{k=1}^{K} r_{t,k} = \frac{6}{8} = \frac{3}{4},$$

$$\text{TD}\Big|_{s=B}: \qquad \sum_{k=1}^{K} r_{t,k} - V(B) = 0 \qquad \Leftrightarrow \qquad V(B) = \frac{1}{K}\sum_{k=1}^{K} r_{t,k} = \frac{6}{8} = \frac{3}{4}.$$

Now consider **state** A, where we have only one trajectory:

- the instantaneous reward following A is zero, and the return of that episode is also zero.
- The TD bootstrap estimate of B is $V(B) = \frac{3}{4}$.

$$\text{MC}\Big|_{s=A}: \qquad \sum_{k=1}^{K} g_{t,k} - V(A) = 0 \qquad \Leftrightarrow \qquad V(A) = 0,$$

$$\text{TD}\Big|_{s=A}: \qquad \sum_{k=1}^{K} r_{t,k} + \gamma V(B) - V(A) = \sum_{k=1}^{K} \gamma V(B) - V(A) = 0 \qquad \Leftrightarrow \qquad V(A) = \gamma V(B) = \frac{3}{4}.$$

Certainty equivalence

Where does this difference come from? $\Rightarrow$ Without going into a detailed derivation:

- MC batch learning converges to the least **squares fit of the sampled returns**:

$$\sum_{k=1}^{K} \sum_{t=1^T} (g_{t,k} - V(s_{t,k}))^2.$$

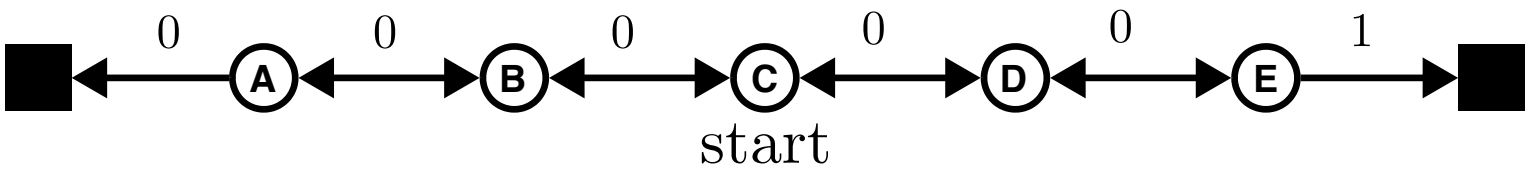
- TD batch learning converges to the **maximum likelihood estimate** such that $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ explains the data with highest probability:

$$\hat{p}(s'|s,a) = \frac{1}{n(s,a)} \sum_{k=1}^{K} \sum_{t=1^T} \mathbb{1}(s_{t+1,k} = s'|s_{t,k} = s, a_{t,k} = a),$$
$$\hat{r} = \frac{1}{n(s,a)} \sum_{k=1}^{K} \sum_{t=1^T} \mathbb{1}(s_{t,k} = s, a_{t,k} = a) r_{t,k}.$$

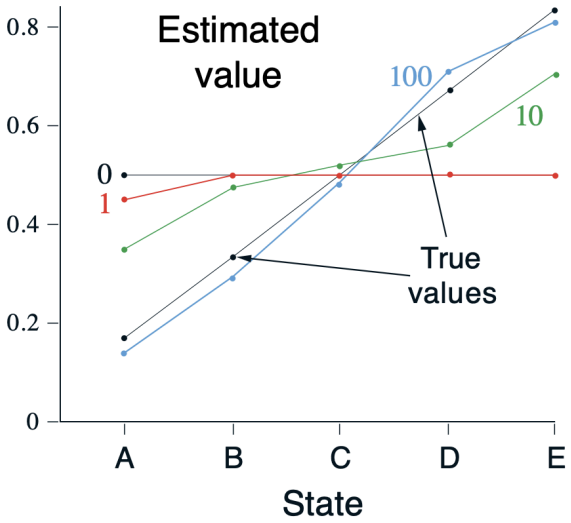
$\Rightarrow$ *TD assumes an MDP problem structure* and is absolutely certain that its internal model concept describes the real world perfectly (so-called **certainty equivalence**).

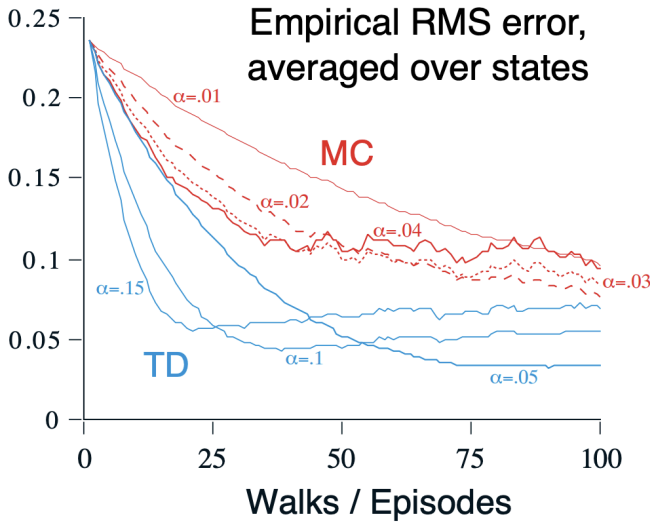
The *random walk* example (Sutton and Barto 1998, Ex. 6.2)

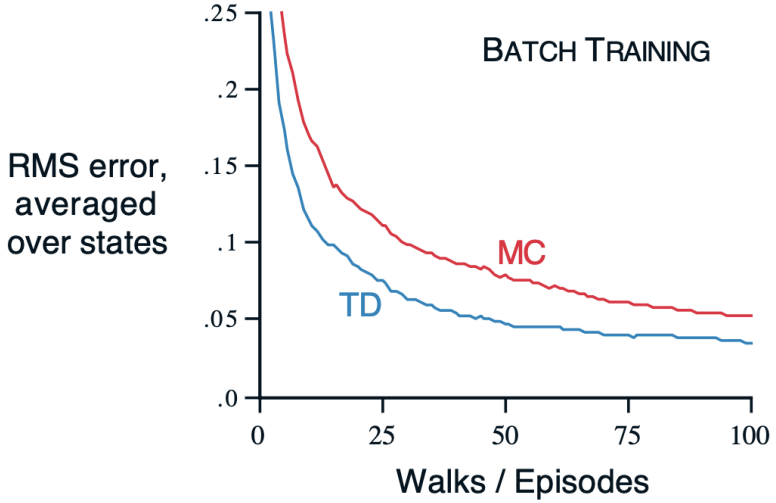
Consider a simple Markov reward process (MRP; no actions), for which we want to estimate $V(s)$ from experience only.



- For all states s, we have $p(\leftarrow) = p(\rightarrow) = 0.5$.
- The MRP is undiscounted ($\gamma = 1$).
- The value of each state is the probability of terminating on the right: $V(A/B/C/D/E) = \frac{1}{6}\ /\ \frac{1}{3}\ /\ \frac{1}{2}\ /\ \frac{2}{3}\ /\ \frac{5}{6}$.







TD(0) learning *for the random walk.* ***Left****: Different numbers of iterations (α=0.1).* ***Middle****: Learning curves for different values of α.* ***Right****: Batch training.*

Policy improvement / control using TD learning

Application of generalized policy iteration to TD learning

The GPI concept can directly be applied to the TD framework by replacing values with action-values, where we alternate between estimation (i.e., prediction) and improvement:

$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \dots \xrightarrow{I} \pi^* \xrightarrow{E} Q^* = Q^{\pi^*}.$$

Estimation: TD(0) update of the Q-function

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left[\overbrace{r_t + \gamma Q(s_{t+1}, a_{t+1})}^{\text{TD target}} - Q(s_t, a_t)\right]}_{\text{TD error } \delta}. \tag{4}$$

The policy: ϵ-greedy / ϵ-soft*.

Convergence (Sutton and Barto 1998)

- towards Q^* with probability one if all tuples (s, a) are visited infinitely often and the step-size condition (3) is satisfied.
- towards π^* if the policy is GLIE.

Key question: how do we select a_{t+1}? $\Rightarrow$ **On-policy versus off policy!**

* $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$.

On-policy TD control: SARSA

s, a, r, s', a': **S**tate-**A**ction-**R**eward-Next **S**tate-Next **A**ction

Algorithm: SARSA.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $k = 1, 2, \ldots, K$ episodes:
  Initialize $s_t \leftarrow s_0, t \leftarrow 0$
  while s_t is not terminal:
    Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})
    Select $a_{t+1} \sim \pi(s_{t+1})$
    Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right]$$

    Update policy: $\pi = \epsilon\text{-greedy}(Q)$ (This is on-policy!)
    $t \leftarrow t + 1$

Convergence: SARSA for finite-state and finite-action MDPs converges to the optimal action-value,

$$Q(s, a) \to Q^*(s, a),$$

under the following conditions:

1. The policy sequence $\pi_t(a|s)$ is GLIE.
2. The step-sizes α_t satisfy the step-size condition (3) (as does, e.g., $\alpha_t = \frac{1}{t}$).

Q-Learning

- SARSA estimates Q of the current policy
- Q-learning is similar but directly estimates Q^*
- off-policy update, since the optimal action-value function is updated independent of a given behavior policy

Algorithm: Q-Learning.
initialize

- $Q(s, a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $Q(\text{terminal-state}, \cdot) = 0$
- $\pi = \epsilon\text{-greedy}(Q)$

for $j = 1, 2, \ldots, J$ episodes:

Initialize $s_t \leftarrow s_0, t \leftarrow 0$

while s_t is not terminal:

Take action $a_t \sim \pi(s_t)$ and observe (r_t, s_{t+1})

Select $a_{t+1} \sim \pi(s_{t+1})$

Update Q given $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Update policy $\pi = \epsilon$-greedy(Q)

$t \leftarrow t + 1$

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.